SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floating-point computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator **Faculty Incharge**

Question 1

Problem

A program adds two signed 8-bit integers:

+120 + +50

The expected mathematical result is:

120 + 50 = 170

However, an 8-bit signed integer can store values only from −128 to +127.

Since 170 is outside this range, the processor produces an incorrect result.

Step 1: Convert Decimal to 8-bit Binary

+120

120 = 01111000

+50

50 = 00110010

+120 = 01111000

+ 50 = 00110010

Step 2: Perform Binary Addition

Carry : 11110000

↓↓↓↓

01111000 + 00110010 = 10101010

Result obtained:

10101010Step 3: Interpret the Result

Since the MSB (Most Significant Bit) is 1, the processor treats the number as negative.

Find its decimal value.

Take 2's complement:

10101010

1's Complement

01010101

Add 1

01010110

01010110 = 86

Therefore,

10101010 = -86

Processor Output:

-86

Actual Answer:

170

Hence,

Processor Output ≠ Actual Answer

Step 4: **Debugging the Issue**

Both operands are positive numbers.

+120 +50

But the obtained result is negative.

This indicates Signed Overflow.

Overflow occurs because the actual result (170) is greater than the maximum value that can be stored in an 8-bit signed integer (+127).

Overflow Detection

Overflow in 2's complement occurs when:

- Positive + Positive = Negative
- Negative + Negative = Positive

Here,

Positive (+120)

Positive (+50)

↓

Negative (-86)

Therefore,

Overflow has occurred.

Why Overflow Occurred

The signed 8-bit range is

-128 to +127

But

170 > +127

Hence the extra carry cannot be represented within 8 bits.

The processor keeps only the lower 8 bits, producing an incorrect result.

Cause of Error

The program does not check whether the addition result exceeds the allowable range of an 8-bit signed integer.

As a result, the overflow is ignored, and an incorrect negative value is produced.

Solution

The processor should detect overflow after every signed addition.

Method 1: Check Operand Signs

If

- Both operands are positive and the result is negative → Overflow.
- Both operands are negative and the result is positive → Overflow.

Method 2: Compare Carry Bits

Overflow occurs if:

Carry into MSB ≠ Carry out of MSB

If both carries are different, set the Overflow Flag (V).

Method 3: Use Larger Data Type

Instead of 8-bit integers, use:

- 16-bit integer
- 32-bit integer

Example:

120 + 50 = 170

A 16-bit signed integer can easily store 170, so overflow does not occur.

Optimized Solution

1. Perform binary addition.
2. Check the overflow condition.
3. If overflow occurs:
    - Set the Overflow Flag.
    - Display an error message or use a larger integer type.
4. Otherwise, store the result normally.

Conclusion

The incorrect result occurs because the sum (+120 + +50 = 170) exceeds the 8-bit signed integer range (−128 to +127). During binary addition, the processor stores only the lower 8 bits, producing 10101010 (−86) instead of 170. The overflow can be detected by checking the operand signs or comparing the carry into and out of the most significant bit. Using overflow detection or a larger integer size prevents incorrect results

Question 2

Problem

A processor performs binary addition using a Ripple Carry Adder (RCA).

During arithmetic operations, the processor experiences high delay because each carry must wait for the previous carry before it can be calculated.

This delay affects the overall speed of the processor, especially in real-time applications.

Example

Consider two 4-bit binary numbers:

A = 1011 (11)

B = 0101 (5)

Expected Result:

11 + 5 = 16

Binary = 10000

Step 1: Ripple Carry Addition

```
   Carry
C4 C3 C2 C1 C0
   1  0  1  1
+  0  1  0  1
----------------
1  0  0  0  0
```

Result:

10000

Step 2: Carry Propagation

Carry generation occurs one bit at a time.

Bit 0

1 + 1 = 10

Sum = 0

Carry = 1

↓

Bit 1

1 + 0 + Carry(1) = 10

Sum = 0

Carry = 1

↓

Bit 2

0 + 1 + Carry(1) = 10

Sum = 0

Carry = 1

↓

Bit 3

1 + 0 + Carry(1) = 10

Sum = 0

Carry = 1

↓

Final Carry

1

Final Result

10000

Step 3: **Debugging the Problem**

In a Ripple Carry Adder, each carry depends on the previous carry.

Bit0

↓

Carry0

↓

Bit1

↓

Carry1

↓

Bit

↓

Carry2

↓

Bit3

↓

Carry3

The carry must ripple through every stage before the final result is obtained.

As the number of bits increases (8-bit, 16-bit, 32-bit, etc.), the delay also increases.

Cause of Performance Bottleneck

The processor waits for each carry to be generated before computing the next bit.

For an n-bit Ripple Carry Adder:

Total Delay ∝ Number of Bits (n)

Therefore,

Higher number of bits → More carry propagation → More execution time.

Solution: Carry Look-Ahead Adder (CLA)

Instead of waiting for carries one by one, a Carry Look-Ahead Adder (CLA) calculates all carry values simultaneously.

It uses two signals:

Generate (G)

$G_i = A_i \times B_i$

A carry is generated when both input bits are 1.

Propagate (P)

$P_i = A_i \oplus B_i$

A carry is propagated when only one input bit is 1.Carry Equations

$C_1 = G_0 + P_0C_0$

$C_2 = G_1 + P_1G_0 + P_1P_0C_0$

$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$

$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

All carries are calculated at the same time, eliminating the ripple effect.

Working of Carry Look-Ahead Adder

Inputs

↓

Generate (G)

↓

Propagate (P)

↓

Carry Look-Ahead Logic

↓

All Carries Computed Simultaneously

↓

Final Sum

Comparison

Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry propagates one bit at a time	Carries are computed simultaneously
High delay	Very low delay
Simple design	More complex design
Slow for large bit sizes	Fast for large bit sizes
Suitable for small circuits	Suitable for high-speed processors

Optimized Solution

1. Replace the Ripple Carry Adder with a Carry Look-Ahead Adder.
2. Generate Generate (G) and Propagate (P) signals.
3. Compute all carry values in parallel.
4. Produce the final sum without waiting for previous carries.
5. This significantly reduces execution time and improves processor performance.

Conclusion

The performance bottleneck in the Ripple Carry Adder occurs because the carry must propagate sequentially through each bit, increasing delay as the number of bits grows. A Carry Look-Ahead Adder (CLA) solves this problem by using Generate and Propagate signals to calculate all carries in parallel. This reduces propagation delay, increases execution speed, and makes the processor suitable for high-speed and real-

Question 3

Problem

A processor uses Booth's Algorithm to multiply signed binary numbers.

During multiplication of negative operands, the processor produces incorrect results because of errors in:

- Bit-pair checking
- Arithmetic right shifting
- Sign extension

These errors lead to an incorrect final product.

Example

Multiply:

Multiplicand (M) = -6

Multiplier (Q) = +5

Using 4-bit 2's complement:

M = -6 = 101

-M = +6 = 0110

Q = +5 = 0101

A = 0000

Q-1 = 0

Count = 4

Booth's Rules

Q_0 Q_{-1}	Operation
00	No operation
01	A = A + M
10	A = A - M

Q_0 Q_{-1} Operation

11 No operation

After every operation:

Perform Arithmetic Right Shift (ARS).

Step 1

Initial Values

A = 0000

Q = 0101

Q-1 = 0

Count = 4

Q_0Q_{-1}

10

Operation

A = A - M= 0000 – 1010 = 0110

Arithmetic Right Shift

Before Shift

A = 0110

Q = 0101

Q-1 = 0

After Shift

A = 0011

Q = 0010

Q-1 = 1

Count = 3

Step 2

Q_0Q_{-1}

01

Operation

A = A + M

0011 + 1010=1101

Arithmetic Right Shift

Before

A = 1101

Q = 0010

Q-1 = 1

After Shift

A = 1110

Q = 1001

Q-1 = 0

Count = 2

Step 3

Q_0Q_{-1}

10

Operation

A = A - M

1110 - 1010=0100

Arithmetic Right Shift

Before

A = 010

Q = 1001

Q-1 = 0

After Shift

A = 0010

Q = 0100

Q-1 = 1

Count = 1

Step 4

Q_0Q_{-1}

01

Operation

A = A + M

0010 + 1010=1100

Arithmetic Right Shift

Before

A = 1100

Q = 0100

Q-1 = 1

After Shift

A = 1110

Q = 0010

Q-1 = 0

Count = 0

Final Product

Combine A and Q

A = 1110

Q = 0010

Product = 11100010

Decimal Value

11100010 = -30

Verification

-6 × 5 = -30

Hence,

Product is Correct.

Debugging the Algorithm

The multiplication result becomes incorrect if any of the following errors occur.

1. Incorrect Bit-Pair Checking

Wrong decision for:

Q_0Q_{-1} - 00 – 01 – 10 - 11

2. Incorrect Arithmetic Right Shift

During arithmetic shift, the sign bit (MSB) must be copied.

Correct

111

↓

1111

Wrong

1110

↓

0111

If the sign bit is not preserved, negative numbers become positive, causing incorrect multiplication.

3. Incorrect Sign Extension

Negative numbers must retain the MSB during every shift.

Correct

1110

↓

1111

Incorrect

1110

↓

0111

Improper sign extension changes the value of the number.

4. Incorrect Count Value

The loop must execute exactly n times, where n is the number of bits.

Too few or too many iterations result in an incorrect product.

Cause of Error

The multiplication unit produces incorrect results because:

- Bit-pair checking is incorrect.
- Arithmetic right shift is not performed properly.
- Sign extension is missing.
- The iteration count is incorrect.

Solution

To ensure correct signed multiplication:

1. Check the bit pair (Q_0Q_{-1}) correctly in every iteration.
2. Perform the correct operation (A + M, A − M, or no operation).
3. Use Arithmetic Right Shift, preserving the sign bit.
4. Maintain proper sign extension after every shift.
5. Repeat the process exactly n times.

Advantages of Correct Booth's Algorithm

- Correctly multiplies positive and negative numbers.
- Reduces the number of addition and subtraction operations.

- Increases multiplication efficiency.
- Produces accurate signed multiplication results.

Conclusion

The incorrect multiplication result is caused by errors in bit-pair checking, arithmetic right shifting, or sign extension. By correctly applying Booth's rules, preserving the sign bit during every arithmetic shift, and executing the algorithm for the required number of iterations, the processor produces the correct signed multiplication result. In the above example, $-6 \times +5 = -30$, confirming the correct operation of Booth's Algorithm

Question 4

An embedded system performs binary division using the Restoring Division Algorithm.

The processor takes more execution time because whenever subtraction produces a negative remainder, it restores the previous value before continuing.

These repeated restoration operations reduce the overall performance.

Example

Divide:

Dividend = 13

Divisor = 3

Binary Representation (5 bits)

Dividend (Q) = 01101

Divisor (M) = 00011

Accumulator (A) = 00000

Count = 5

Step 1

Left Shift (A,Q)

A = 00000

Q = 11010

Subtract Divisor

A = 00000 – 00011 = 11101

Since A is negative, restore A.

A = 11101 + 00011 = 00000

Set

$Q_0 = 0$

Count = 4

Step 2

Left Shift

A = 00001

Q = 10100

Subtract Divisor

00001 – 00011 = 11110

Negative Result

Restore

11110 + 00011 = 00001

Set

$Q_0 = 0$

Count = 3

Step 3

Left Shift

A = 00011

Q = 01000

Subtract Divisor

00011 – 00011=00000

Positive Result

Set

$Q_0 = 1$

Count = 2

Step 4

Left Shift

A = 00000

Q = 10010

Subtract Divisor

00000 – 00011 =11101

Negative Result

Restore

11101 + 00011= 00000

Set

$Q_0 = 0$

Count = 1

Step 5

Left Shift

A = 00001

Q = 00100

Subtract Divisor

00001 – 00011 = 11110

Negative Result

Restore

11110 + 00011=00001

Set

$Q_0 = 0$

Count = 0

Final Result

Quotient = 00100 = 4

Remainder = 00001 = 1

Verification

13 ÷ 3 = 4

Remainder = 1

Debugging the Problem

The algorithm performs the following operations repeatedly:

Subtract

↓

Negative Result

↓

Restore

↓

Continue

Every negative remainder requires an additional restoration (addition) operation.

These extra operations increase the execution time.

Cause of Inefficiency

The restoring division algorithm always restores the previous remainder whenever subtraction produces a negative result.

Subtract

↓

Negative

↓

Restore

↓

Next Step

Thus, unnecessary restoration operations make the algorithm slower.

Optimized Solution: Non-Restoring Division Algorithm

Instead of restoring immediately, the Non-Restoring Division Algorithm does not restore the negative remainder.

It keeps the negative value and performs the opposite operation in the next iteration.

Working

If remainder is positive

Subtract Divisor

If remainder is negative

Add Divisor

No restoration is performed after every negative result.

Workflow of Non-Restoring Division

Left Shift

↓

Check Sign of Remainder

↓

Positive → Subtract Divisor

Negative → Add Divisor

↓

Set Quotient Bit

↓

Repeat

Only after the final iteration, if the remainder is negative, one final correction is made.

Comparison

Restoring Division	Non-Restoring Division
Restores after every negative subtraction	No immediate restoration
More addition operations	Fewer operations

Restoring Division	Non-Restoring Division
Higher execution time	Faster execution
Lower efficiency	Higher efficiency
Suitable for simple systems	Suitable for high-speed processors

Advantages of Non-Restoring Division

- Eliminates unnecessary restoration steps.
- Reduces the number of arithmetic operations.
- Improves processor speed.
- Increases overall efficiency.
- Better suited for embedded and real-time systems.

Cause of Error

The embedded system becomes slow because the restoring division algorithm performs repeated restoration whenever the remainder becomes negative.

These unnecessary restoration operations increase the total execution time.

Solution

1. Replace the Restoring Division Algorithm with the Non-Restoring Division Algorithm.
2. If the remainder is positive, subtract the divisor.
3. If the remainder is negative, add the divisor in the next iteration instead of restoring immediately.
4. Perform only one final correction if the remainder is negative after the last iteration.
5. This reduces unnecessary arithmetic operations and improves execution speed.

Conclusion

The inefficiency in the restoring division algorithm is caused by repeated restoration operations after every negative remainder. These extra additions increase execution time and reduce processor performance. The Non-Restoring

Division Algorithm eliminates these unnecessary restoration steps by carrying the negative remainder to the next iteration and performing the opposite operation only when needed. As a result, it requires fewer arithmetic operations, executes faster, and is more suitable for embedded and real-time systems.

Question 5

Problem

A scientific application performs floating-point addition using the IEEE 754 Single Precision (32-bit) format.

When a very large number and a very small number are added together, the result becomes inaccurate because the small number may be lost during the addition process.

Example

Add:

Large Number = 100000000

Small Number = 1

Expected Mathematical Result

100000000 + 1 = 100000001

However, in IEEE 754 Single Precision, the processor may store the result as:

100000000

The 1 is lost due to limited precision.

IEEE 754 Single Precision Format

A 32-bit floating-point number consists of:

1 Bit → Sign

8 Bits → Exponent

23 Bits → Mantissa (Fraction)

Diagram

| Sign | Exponent (8) | Mantissa (23) |

Step 1: Convert Numbers to Floating-Point Form

Large Number

$100000000 \approx 1.01111101011110000100000 \times 2^{26}$

Small Number $1 = 1.0 \times 2^{0}$

The exponents are different.

Large Exponent = 26

Small Exponent = 0

Step 2: Exponent Alignment

Before addition, both numbers must have the same exponent.

The smaller exponent is shifted right.

1.0×2^{0}

↓

$0.00000000000000000000001 \times 2^{26}$

After repeated right shifts, many significant bits of the small number are discarded because only 23 mantissa bits are available.

Step 3: Addition

Now both numbers have the same exponent.

Large Number

$1.01111101011110000100000 +$

Small Number

$0.00000000000000000000000 = 1.01111101011110000100000$

The small number has no effect on the result.

Step 4: Normalization

After addition, the result is normalized.

Example

10.111010101

↓

$1.0111010101 \times 2^{1}$

Normalization ensures that the mantissa starts with 1.

Step 5: Rounding

Since only 23 mantissa bits can be stored, extra bits are removed.

Debugging the Problem

The inaccuracy occurs because of the following reasons:

1. Exponent Alignment

To match exponents, the mantissa of the smaller number is shifted many times.

As a result,

Small Number

↓

Very Small Value

↓

Lost During Addition

2. Limited Mantissa

Single precision stores only 23 mantissa bits.

Extra bits are discarded.

This reduces accuracy.

3. Rounding Error

After normalization, extra bits are rounded.

Repeated rounding causes cumulative errors.

4. Loss of Precision

Adding a very small number to a very large number causes the smaller value to disappear.

Example

100000000 + 1

↓

100000000

The 1 is not represented in the final result.

Cause of Error

The floating-point result becomes inaccurate because:

- Exponent alignment shifts the smaller number.
- The mantissa has limited precision.
- Rounding removes extra bits.
- Precision is lost during floating-point addition.

Optimized Solution

Method 1: Use Double Precision

Instead of 32-bit Single Precision, use 64-bit Double Precision.

Comparison

Single Precision	Double Precision
32 bits	64 bits
23 Mantissa bits	52 Mantissa bits
Lower accuracy	Higher accuracy
More rounding error	Less rounding error

Double precision preserves more significant digits and reduces precision loss.

Method 2: Rearrange Calculations

Instead of directly adding very small numbers to very large numbers:

Large + Small + Small

Group similar-sized values first:

(Small + Small) + Large

This minimizes rounding errors.

Method 3: Use Accurate Summation Algorithms

Use algorithms such as Kahan Summation, which compensate for small rounding errors during repeated additions.

This improves numerical accuracy in scientific applications.

Method 4: Reduce Unnecessary Rounding

Perform calculations with higher precision internally and round only the final result.

This reduces cumulative rounding errors.

Advantages of the Optimized Solution

- Higher numerical accuracy.
- Reduced rounding errors.
- Better handling of very large and very small numbers.
- Improved reliability in scientific and engineering applications.
- More accurate floating-point calculations.

Conclusion

The inaccurate result in IEEE 754 Single Precision occurs because exponent alignment shifts the smaller number, the 23-bit mantissa cannot store all significant digits, and rounding removes extra bits. These factors lead to a loss of precision when adding numbers with very different magnitudes. The problem can be minimized by using 64-bit Double Precision, rearranging calculations, applying accurate summation algorithms such as Kahan Summation, and reducing unnecessary rounding. These techniques improve the accuracy and reliability of floating-point computations.